

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



AAT REPORT
on

OPERATING SYSTEMS
23CS4PCOPS

Submitted by

Navneet Kumar Dubey(1BF24CS194)

in partial fulfilment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous College Affiliated to VTU)

BENGALURU-560019

AY 2025-2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Autonomous College Affiliated to VTU)

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the AAT work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Navneet Kumar Dubey (1BF24CS194), who is bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the AY 2025-2026. The AAT report has been approved as it satisfies the academic requirements in respect of OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Sandhya A Kulkarni
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin	4-18
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	19-23
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	24-27
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem Write a C program to simulate:	28-32
5.	a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	33-38
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	39-42
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	43-46

Course Outcomes:

CO1	Apply the different concepts and functionalities of Operating Systems.
CO2	Analyse various Operating system strategies and techniques
CO3	Demonstrate the different functionalities of Operating Systems.
CO4	Conduct practical experiments to implement the functionalities of Operating systems.

Lab Program 1:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) FCFS b) SJF c) Priority d) Round Robin

a) FCFS:

```
#include <stdio.h>

int main() {
    int n, i;
    int at[20], bt[20], ct[20], tat[20], wt[20], rt[20];
    float avg_wt=0, avg_tat=0, avg_rt = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Arrival Time and Burst Time:\n");

    for(i=0; i<n; i++){
        printf("P%d AT: ", i+1);
        scanf("%d", &at[i]);

        printf("P%d BT: ", i+1);
        scanf("%d", &bt[i]);
    }

    ct[0] = at[0] + bt[0];

    for(i=1; i<n; i++){
        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }

    for(i=0; i<n; i++){
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        rt[i] = wt[i];

        avg_wt += wt[i];
        avg_tat += tat[i];
    }
}
```

```

    avg_rt += rt[i];
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
}

printf("\nAverage Waiting Time = %.2f",
    avg_wt/n); printf("\nAverage Turnaround Time =
    %.2f", avg_tat/n); printf("\nAverage Response
    Time = %.2f\n", avg_rt/n);

return 0;
}

```

Output:

```

C:\Users\Student\Documents' X + v
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 5 4
Enter Arrival Time and Burst Time for P2: 0 3
Enter Arrival Time and Burst Time for P3: 1 5
Enter Arrival Time and Burst Time for P4: 2 8

FCFS process
Process AT      BT      CT      TAT      WT      RT
P1      5      4      20      15      11      11
P2      0      3      3       3       0       0
P3      1      5      8       7       2       2
P4      2      8      16      14       6       6

Process returned 0 (0x0)   execution time : 14.042 s
Press any key to continue.
|

```

b) SJF:

```
#include<stdio.h>
```

```
#include<limits.h>
```

```
void sjf_non_preemptive(int at[],int bt[],int n){
```

```
    int wt[n],tat[n],rt[n],ct[n],done[n];
```

```
    int time=0,completed=0;
```

```
    for(int i=0;i<n;i++) done[i]=0;
```

```
    while(completed<n){
```

```
        int min=INT_MAX,idx=-1;
```

```
        for(int i=0;i<n;i++){
```

```
            if(at[i]<=time && done[i]==0 && bt[i]<min){
```

```
                min=bt[i];
```

```
                idx=i;
```

```
            }
```

```
        }
```

```
        if(idx==-1){
```

```
            time++;
```

```
            continue;
```

```
        }
```

```
        rt[idx]=time-at[idx];
```

```
        ct[idx]=time+bt[idx];
```

```
        tat[idx]=ct[idx]-at[idx];
```

```
        wt[idx]=tat[idx]-bt[idx];
```

```
        time=ct[idx];
```

```
        done[idx]=1;
```

```
        completed++;
```

```
    }
```

```
    float avgwt=0,avgtat=0,avgrt=0;
```

```
    printf("\nSJF Non Preemptive\n");
```

```
    printf("P\tAT\tBT\tWT\tTAT\tRT\n");
```

```
    for(int i=0;i<n;i++){
```

```
        avgwt+=wt[i];
```

```
        avgtat+=tat[i];
```

```
        avgrt+=rt[i];
```

```
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],wt[i],tat[i],rt[i]);
```

```
    }
```

```
    printf("Average WT : %.2f\n",avgwt/n);
```

```

printf("Average TAT : %.2f\n",avgtat/n);
printf("Average RT : %.2f\n",avgrt/n);
}

void sjf_preemptive(int at[],int bt[],int n){
    int rt[n],wt[n],tat[n],ct[n],start[n];
    int time=0,completed=0;

    for(int i=0;i<n;i++){
        rt[i]=bt[i];
        start[i]=-1;
    }

    while(completed<n){
        int min=INT_MAX,idx=-1;

        for(int i=0;i<n;i++){
            if(at[i]<=time && rt[i]>0 && rt[i]<min){
                min=rt[i];
                idx=i;
            }
        }

        if(idx==-1){
            time++;
            continue;
        }

        if(start[idx]==-1)
            start[idx]=time;

        rt[idx]--;
        time++;

        if(rt[idx]==0){
            completed++;
            ct[idx]=time;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];
        }
    }

    float avgwt=0,avgtat=0,avgrt=0;

    printf("\nSJF Preemptive (SRTF)\n");
    printf("P\tAT\tBT\tWT\tTAT\tRT\n");

    for(int i=0;i<n;i++){
        int response=start[i]-at[i];

        avgwt+=wt[i];
        avgtat+=tat[i];
    }
}

```

```

    avgrt+=response;

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",i+1,at[i],bt[i],wt[i],tat[i],response);
}

printf("Average WT : %.2f\n",avgwt/n);
printf("Average TAT : %.2f\n",avgtat/n);
printf("Average RT : %.2f\n",avgrt/n);
}

int main(){
    int n,choice;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    int at[n],bt[n];

    for(int i=0;i<n;i++){
        printf("Enter AT of P%d: ",i+1);
        scanf("%d",&at[i]);

        printf("Enter BT of P%d: ",i+1);
        scanf("%d",&bt[i]);
    }

    printf("\n1. Preemptive SJF\n2. Non Preemptive SJF\n");
    printf("Enter choice: ");
    scanf("%d",&choice);

    if(choice==1)
        sjf_preemptive(at,bt,n);
    else if(choice==2)
        sjf_non_preemptive(at,bt,n);
    else
        printf("Invalid choice");

    return 0;
}

```

Output:

```
C:\Users\Student\Documents' X + v
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 4
Enter Arrival Time and Burst Time for P2: 1 5
Enter Arrival Time and Burst Time for P3: 2 7
Enter Arrival Time and Burst Time for P4: 4 6

1. SJF Non-Preemptive
2. SJF Preemptive (SRTF)
Enter choice: 2

SJF Preemptive (SRTF) Result:
Process AT    BT    CT    TAT    WT    RT
P1      0      4      4      4      0      0
P2      1      5      9      8      3      3
P3      2      7     22     20     13     13
P4      4      6     15     11      5      5

Process returned 0 (0x0)  execution time : 11.708 s
Press any key to continue.
|
```

```
C:\Users\Student\Documents' X + v
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 4
Enter Arrival Time and Burst Time for P2: 1 4
Enter Arrival Time and Burst Time for P3: 2 6
Enter Arrival Time and Burst Time for P4: 4 8

1. SJF Non-Preemptive
2. SJF Preemptive (SRTF)
Enter choice: 1

SJF Non-Preemptive Result:
Process AT    BT    CT    TAT    WT    RT
P1      0      4      4      4      0      0
P2      1      4      8      7      3      3
P3      2      6     14     12      6      6
P4      4      8     22     18     10     10

Process returned 0 (0x0)  execution time : 18.055 s
Press any key to continue.
|
```

c) Priority(Preemptive):

```
#include <stdio.h>
#include <limits.h>

int main()
{
    int n,i,time=0,completed=0;
    int
    at[20],bt[20],pr[20],rem_bt[2
    0]; int
    ct[20],tat[20],wt[20],rt[20];

    float avg_wt=0,avg_tat=0,avg_rt=0;

    printf("Enter number of
    processes: "); scanf("%d",&n);

    printf("Enter Arrival Time, Burst Time and Priority:\n");

    for(i=0;i<n;i++)
    {
        printf("P%d AT: ",i+1);
        scanf("%d",&at[i]);

        printf("P%d BT: ",i+1);
        scanf("%d",&bt[i]);

        printf("P%d Priority: ",i+1);
        scanf("%d",&pr[i]);

        rem_bt[i]=bt[i];
        rt[i]=-1;
    }

    while(completed<n)
    {
        inthighest=-1;
        intmin_pr=INT_MAX;

        for(i=0;i<n;i++)
```

```

{
    if(at[i]<=time && rem_bt[i]>0 &&
        pr[i]<min_pr) {
        min_pr=pr[i];
        highest=i;
    }
}

if(highest==-1)
{
    time++;
    continue;
}

if(rt[highest]==-1)
    rt[highest] = time - at[highest];

rem_bt[highest]--;
time++;

if(rem_bt[highest]==0)
{
    completed++;

    ct[highest]=time;
    tat[highest]=ct[highest]-at[highest];
    wt[highest]=tat[highest]-bt[highest];

    avg_wt+=wt[highest];
    avg_tat+=tat[highest];
    avg_rt+=rt[highest];
}
}

printf("\nProcess AT BT PR CT TAT WT RT\n");

for(i=0;i<n;i++)
{
    printf("P%d %d %d %d %d %d %d\n",
        i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i],rt[i]);

```

```

}

printf("\nAverage Waiting Time =
%.2f",avg_wt/n); printf("\nAverage Turnaround
Time = %.2f",avg_tat/n); printf("\nAverage
Response Time = %.2f\n",avg_rt/n);

return 0;
}

```

Output:

```

Enter number of processes: 4
Enter AT, BT and Priority for P1: 0 7 3
Enter AT, BT and Priority for P2: 2 4 1
Enter AT, BT and Priority for P3: 4 1 2
Enter AT, BT and Priority for P4: 5 4 2

1. Priority Non-Preemptive
2. Priority Preemptive
Enter choice: 1

Priority Non-Preemptive Result:

```

Process	AT	BT	PR	CT	TAT	WT	RT
P1	0	7	3	7	7	0	0
P2	2	4	1	11	9	5	5
P3	4	1	2	12	8	7	7
P4	5	4	2	16	11	7	7

```

Average TAT = 8.75
Average WT = 4.75
Average RT = 4.75

```

Priority (Non-preemptive):

```

#include <stdio.h>
#include <limits.h>

int main()
{
    int n,i,time=0,completed=0;
    int at[20],bt[20],pr[20];
    int ct[20],tat[20],wt[20],rt[20];
    int done[20]={0};

    float avg_wt=0,avg_tat=0,avg_rt=0;

    printf("Enter number of processes: ");

```

```

scanf("%d",&n);

printf("Enter Arrival Time, Burst Time and Priority:\n");

for(i=0;i<n;i++)
{
    printf("P%d AT: ",i+1);
    scanf("%d",&at[i]);

    printf("P%d BT: ",i+1);
    scanf("%d",&bt[i]);

    printf("P%d Priority: ",i+1);
    scanf("%d",&pr[i]);
}

while(completed<n)
{
    int highest=-1;
    int min_pr=INT_MAX;

    for(i=0;i<n;i++)
    {
        if(at[i]<=time && done[i]==0 &&
        pr[i]<min_pr) {
            min_pr=pr[i];
            highest=i;
        }
    }

    if(highest==-1)
    {
        time++;
        continue;
    }

    rt[highest] = time - at[highest];

    time+=bt[highest];
    ct[highest] = time;

```

```

    tat[highest] = ct[highest] - at[highest];
    wt[highest] = tat[highest] - bt[highest];

    avg_wt+= wt[highest];
    avg_tat+= tat[highest];
    avg_rt+= rt[highest];

    done[highest]=1;
    completed++;
}

printf("\nProcess AT BT PR CT TAT WT RT\n");

for(i=0;i<n;i++)
{
    printf("P%d %d %d %d %d %d %d\n",
        i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i],rt[i]);
}

printf("\nAverage Waiting Time =
%.2f",avg_wt/n); printf("\nAverage Turnaround
Time = %.2f",avg_tat/n); printf("\nAverage
Response Time = %.2f\n",avg_rt/n);

return 0;
}

```

```

Enter number of processes: 4
Enter AT, BT and Priority for P1: 0 7 3
Enter AT, BT and Priority for P2: 2 4 1
Enter AT, BT and Priority for P3: 4 1 2
Enter AT, BT and Priority for P4: 5 4 2

1. Priority Non-Preemptive
2. Priority Preemptive
Enter choice: 1

Priority Non-Preemptive Result:

```

Process	AT	BT	PR	CT	TAT	WT	RT
P1	0	7	3	7	7	0	0
P2	2	4	1	11	9	5	5
P3	4	1	2	12	8	7	7
P4	5	4	2	16	11	7	7

```

Average TAT = 8.75
Average WT = 4.75
Average RT = 4.75

```

Output:

d) Round Robin:

```
#include <stdio.h>

void roundRobin(int n, int at[], int bt[],
    int tq) {
    int rt[n], ct[n], wt[n], tat[n], resp[n];

    for (int i = 0; i < n; i++) {
        rt[i] = bt[i];
        resp[i] = -1;
    }

    int time = 0, completed = 0;

    while (completed < n) {
        int idle = 1;

        for (int i = 0; i < n; i++) {
            if (at[i] <= time && rt[i] > 0) {
                idle = 0;

                if (resp[i] == -1) {
                    resp[i] = time - at[i];
                }

                if (rt[i] > tq) {
                    time += tq;
                    rt[i] -= tq;
                } else {
                    time += rt[i];
                    ct[i] = time;
                    rt[i] = 0;
                    completed++;
                }
            }
        }

        if (idle) {
            time++;
        }
    }
}
```

```
}
```

```
floattotal_wt=0,total_tat= 0, total_rt = 0;
```

```
for(inti =0;i <n;i++) {
```

```
    tat[i] = ct[i] - at[i];
```

```
    wt[i]=tat[i]-bt[i];
```

```
    total_wt += wt[i];
```

```
    total_tat += tat[i];
```

```
    total_rt += resp[i];
```

```
}
```

```
printf("\nTimeQuantum=%d\n", tq);
```

```
printf("Process\tAT\tBT\tCT\tWT\tTAT\tRT\n");
```

```
for(inti =0;i <n;i++) {
```

```
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t\n", i+1,at[i],bt[i],ct[i], wt[i], tat[i],  
    resp[i]); }
```

```
printf("AverageWT=%.2f\n",
```

```
total_wt / n);
```

```
printf("AverageTAT=%.2f\n",
```

```
total_tat / n);
```

16

```
printf("Average RT = %.2f\n",  
total_rt / n); }
```

```
intmain() {
```

```
    intn;
```

```
printf("Enter number of processes: ");
```

```
scanf("%d", &n);
```

```
intat[n], bt[n];
```

```
for(int i = 0; i < n; i++) {
```

```
    printf("\nProcess P%d\n", i + 1);
```

```
    printf("Arrival Time: ");
```

```
    scanf("%d", &at[i]);
```

```

    printf("Burst Time: ");
    scanf("%d", &bt[i]);
}

int num_q, tq;

printf("\nEnter number of different Time
Quantums: "); scanf("%d", &num_q);

for(int i = 0; i < num_q; i++) {
    printf("\nEnter Time Quantum %d: ",
    i + 1); scanf("%d", &tq);

    roundRobin(n, at, bt, tq);
}

return 0;
}

```

Output:

```

Enter number of processes: 4
Enter AT and BT for P1: 0 5
Enter AT and BT for P2: 2 4
Enter AT and BT for P3: 0 6
Enter AT and BT for P4: 5 8
Enter Time Quantum: 3

Round Robin Result:

```

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	11	11	6	0
P2	2	4	18	16	12	4
P3	0	6	17	17	11	3
P4	5	8	23	18	10	6

```

Average TAT = 15.50
Average WT = 9.75
Average RT = 3.25

```

Lab Program 2:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>
#include <string.h>
#define MAX_PROCESSES 100

typedef struct {
    int process_id;
    int arrival_time;
    int burst_time;
    int remaining_time;
    int type;
    int completion_time;
    int waiting_time;
    int turnaround_time;
    int started;
    int start_time;
} Process;

typedef struct {
    int items[MAX_PROCESSES];
    int front, rear;
} Queue;

void initQueue(Queue* q) {
    q->front = 0;
    q->rear = -1;
}

int isEmpty(Queue* q) {
    return q->rear < q->front;
}

void enqueue(Queue* q, int value) {
    if (q->rear < MAX_PROCESSES - 1) {
        q->rear++;
        q->items[q->rear] = value;
    }
```

```
}
```

```
int dequeue(Queue* q) {
    if (!isEmpty(q)) {
        int val = q->items[q->front];
        q->front++;
        return val;
    }
    return -1;
}
```

```
int compareArrival(const void* a, const
void* b) { Process* p1 = (Process*)a;
Process* p2 = (Process*)b;
if (p1->arrival_time < p2->arrival_time)
    return -1;
else if (p1->arrival_time > p2->arrival_time)
    return 1;
else
    return 0;
}
```

```
int main() {
    int n;
    Process processes[MAX_PROCESSES];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("\nProcess %d\n", i);
        printf("Enter arrival time: ");
        scanf("%d", &processes[i].arrival_time);
        printf("Enter burst time: ");
        scanf("%d", &processes[i].burst_time);
        printf("Enter type (0 = System, 1 =
User): "); scanf("%d",
&processes[i].type);

        processes[i].process_id = i;
        processes[i].remaining_time =
```

```

processes[i].burst_time;
processes[i].completion_time = 0;
processes[i].waiting_time = 0;

processes[i].turnaround_time = 0;
processes[i].started = 0;
processes[i].start_time=-1;
}

qsort(processes,n,sizeof(Process), compareArrival);

QueuesystemQueue,userQueue;
initQueue(&systemQueue);
initQueue(&userQueue);

int completed = 0;
int current_time = 0;
int i = 0;
Process*current_process=NULL;

while (completed < n) {
    while(i<n&&processes[i].arrival_time <=
        current_time) { if(processes[i].type==0)
        enqueue(&systemQueue, i);
        else
        enqueue(&userQueue, i);
        i++;
    }

    if(current_process!=NULL) {
        if(current_process->type == 1 &&
            !isEmpty(&systemQueue)) { enqueue(&userQueue,
            current_process - processes); current_process=NULL;
        }
    }

    if(current_process==NULL) {
        if(!isEmpty(&systemQueue)) {
            intidx=dequeue(&systemQueue);
            current_process=&processes[idx];
        }elseif(!isEmpty(&userQueue)) {
            intidx=dequeue(&userQueue);

```

```

        current_process=&processes[idx];
    } else {
        current_time++;
        continue;
    }
}

if(current_process->started == 0) {
    current_process->start_time = current_time;
    current_process->started = 1;
}

current_process->remaining_time--;
current_time++;

if(current_process->remaining_time == 0) {
    current_process->completion_time = current_time;
    current_process->turnaround_time = current_process->completion_time -
current_process->arrival_time;
    current_process->waiting_time = current_process->turnaround_time -
    current_process->burst_time; completed++;
    current_process = NULL;
}
}

printf("\nID\tType\tAT\tBT\tCT\tWT\tTAT\n");
double total_wt = 0, total_tat = 0;
for(int j= 0; j < n; j++) {
    printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
        processes[j].process_id,
        (processes[j].type == 0) ? "System" : "User",
        processes[j].arrival_time,
        processes[j].burst_time,
        processes[j].completion_time,
        processes[j].waiting_time,
        processes[j].turnaround_time);

    total_wt += processes[j].waiting_time;
    total_tat += processes[j].turnaround_time;
}

```

```

printf("\nAverage Waiting Time: %.2f\n", total_wt / n);
printf("Average Turnaround Time: %.2f\n", total_tat / n);

return 0;
}

```

22

Output:

```

Enter number of processes: 4
Enter AT, BT and Type (0=System, 1=User)
For process 1: 0 5 0
For process 2: 2 3 1
For process 3: 1 4 0
For process 4: 3 2 1

ID      Type      AT      BT      CT      WT      TAT
1       System    0       5       5       0       5
2       User      2       3      12       7      10
3       System    1       4       9       4       8
4       User      3       2      14       9      11

Average Waiting Time: 5.00
Average Turnaround Time: 8.50

```

Lab Program 3:

Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling

```
#include <stdio.h>
#include <math.h>

#define MAX 10

typedef struct {
    int id;
    int burst;
    int deadline;
    int period;
    int weight;
    int ct, wt, tat;
} Process;

void edf(Process p[], int n) {
    float util = 0;

    for (int i = 0; i < n; i++)
        util += (float)p[i].burst / p[i].deadline;

    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (p[i].deadline > p[j].deadline) {
                Process t = p[i]; p[i] = p[j]; p[j] = t;
            }

    int time = 0;
    for (int i = 0; i < n; i++) {
        time += p[i].burst;
        p[i].ct = time;
        p[i].tat = p[i].ct;
        p[i].wt = p[i].tat - p[i].burst;
    }

    printf("\n==== EDF Scheduling =====\n");
    printf("Utilization = %.2f\n", util);
    printf("Schedulable if U <= 1\n");
```

```

printf("ID\tBT\tDL\tCT\tWT\tTA
T\n"); for(inti =0;i <n;i++)
    printf("%d\t%d\t%d\t%d\t%d\t
    %d\n",
    p[i].id,p[i].burst,p[i].deadline,
    p[i].ct, p[i].wt, p[i].tat);
}

void rms(Process p[], int n) {
    float util = 0;

    for(inti =0;i <n;i++)
        util+=(float)p[i].burst/p[i].period;

    floatbound=n*(pow(2,(float)1/n) - 1);

    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if(p[i].period>p[j].period) {
                Processt=p[i];p[i]=p[j]; p[j] = t;
            }

    int time =0;
    for(inti =0;i <n;i++) {
        time += p[i].burst;
        p[i].ct = time;
        p[i].tat = p[i].ct;
        p[i].wt=p[i].tat-p[i].burst;
    }

    printf("\n====RMSScheduling
    ====\\n");
    printf("Utilization=%.2f\\n",util);
    printf("RMBound=%.4f\\n",bound);

    if (util <= bound)
        printf("Schedulable\\n");
    else
        printf("Notguaranteedschedulable\\n");
}

```

```
printf("ID\tBT\tPeriod\tCT\tWT\tTA
T\n"); for(inti =0;i <n;i++)
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
        p[i].id, p[i].burst, p[i].period,
        p[i].ct, p[i].wt, p[i].tat);
}
```

```
void proportional(Process p[], int n) {
    int total_weight = 0;

    for(inti =0;i <n;i++)
        total_weight += p[i].weight;

    printf("\n====ProportionalScheduling
====\n");
    printf("TotalWeight=%d\n",total_weight);

    printf("ID\tWeight\tCPU Share\n");

    for(inti =0;i <n;i++) {
        floatshare=(float)p[i].weight/total_weight;
        printf("%d\t%d\t%.2f\n",p[i].id,p[i].weight,
        share); }
}
```

```
int main() {
    int n;
    Processp1[MAX],p2[MAX],p3[MAX];

    printf("Enternumberofprocesses:");
    scanf("%d", &n);

    for(inti =0;i <n;i++) {
        printf("\nProcess%d\n",i);

        p1[i].id = p2[i].id = p3[i].id = i;

        printf("Burst Time: ");
        scanf("%d", &p1[i].burst);
```

```
printf("Deadline (EDF): ");  
scanf("%d", &p1[i].deadline);
```

```
printf("Period (RMS): ");  
scanf("%d", &p1[i].period);
```

26

```
printf("Weight  
(Proportional): ");  
scanf("%d",  
&p1[i].weight);
```

```
p2[i] = p1[i];  
p3[i] = p1[i];  
}
```

```
edf(p1, n);  
rms(p2, n);  
proportional(p3, n);
```

```
return 0;  
}
```

Output:

```
gama@ubuntu:~/Desktop/121-OS-LAB-temp-8-edu/OS-LAB/gama/Desktop/121-OS-LAB$
Enter number of processes: 3

Enter process details:

Process 0:
Burst Time: 3
Deadline (for EDF): 7
Period (for RMS): 20

Process 1:
Burst Time: 2
Deadline (for EDF): 4
Period (for RMS): 5

Process 2:
Burst Time: 2
Deadline (for EDF): 8
Period (for RMS): 10

==== Earliest Deadline First (EDF) Scheduling ====
CPU Utilization: 0.75
Schedulable (Utilization <= 1)


| ID | BF | Deadline | CT | WT | TAT |
|----|----|----------|----|----|-----|
| 0  | 3  | 7        | 5  | 2  | 5   |
| 1  | 2  | 4        | 2  | 0  | 2   |
| 2  | 2  | 8        | 7  | 5  | 7   |



==== Rate Monotonic Scheduling (RMS) ====
CPU Utilization: 0.75
RM Bound: 0.7798
Schedulable (Utilization <= RM Bound)


| ID | BF | Period | CT | WT | TAT |
|----|----|--------|----|----|-----|
| 0  | 3  | 20     | 7  | 4  | 7   |
| 1  | 2  | 5      | 2  | 0  | 2   |
| 2  | 2  | 10     | 4  | 2  | 4   |



==== Proportional Scheduling ====
P0 -> 57.14%
P1 -> 14.29%
P2 -> 28.57%
```

Lab Program 4:

Write a C program to simulate:

a) Producer-Consumer problem using semaphores b) Dining-Philosopher's

problem

a) Producer-Consumer problem using semaphores

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 15

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    for (int i = 0; i < ITEMS; i++) {

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d at buffer[%d]\n", i, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        usleep(100000);
    }
    return NULL;
}

void* consumer(void* arg) {
    for (int i = 0; i < ITEMS; i++) {
```

```
sem_wait(&full);

pthread_mutex_lock(&mutex);

int item = buffer[out];
printf("Consumed: %d from buffer[%d]\n",
item, out); out = (out + 1) % BUFFER_SIZE;

pthread_mutex_unlock(&mutex);
sem_post(&empty);

usleep(150000);
}
return NULL;
}

int main() {
    pthread_t p, c;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer,
    NULL); pthread_create(&c, NULL,
    consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

Output:

```
"C:\Users\maany\OneDrive\D  × + v
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Consumed: 2 from buffer[2]
Produced: 3 at buffer[3]
Consumed: 3 from buffer[3]
Produced: 4 at buffer[4]
Consumed: 4 from buffer[4]
Produced: 5 at buffer[0]
Consumed: 5 from buffer[0]
Produced: 6 at buffer[1]
Consumed: 6 from buffer[1]
Produced: 7 at buffer[2]
Consumed: 7 from buffer[2]
Produced: 8 at buffer[3]
Consumed: 8 from buffer[3]
Produced: 9 at buffer[4]
Consumed: 9 from buffer[4]
Produced: 10 at buffer[0]
Consumed: 10 from buffer[0]
Produced: 11 at buffer[1]
Consumed: 11 from buffer[1]
Produced: 12 at buffer[2]
Consumed: 12 from buffer[2]
Produced: 13 at buffer[3]
Consumed: 13 from buffer[3]
Produced: 14 at buffer[4]
Consumed: 14 from buffer[4]

Process returned 0 (0x0)    execution time : 15.973 s
Press any key to continue.
```

b) Dining-Philosopher's problem

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5 // fewer philosophers

sem_t forks[N];

void* philosopher(void* num) {
    int id = *(int*)num;

    printf("Philosopher %d is thinking.\n", id);

    // Deadlock prevention: last philosopher picks right first
    if (id == N - 1) {
        sem_wait(&forks[(id + 1) % N]); // right fork
        printf("Philosopher %d picked up right fork %d.\n", id, (id + 1) % N);

        sem_wait(&forks[id]); // left fork
        printf("Philosopher %d picked up left fork %d.\n", id, id);

    } else {
        sem_wait(&forks[id]); // left fork
        printf("Philosopher %d picked up left fork %d.\n", id, id);

        sem_wait(&forks[(id + 1) % N]); // right fork
        printf("Philosopher %d picked up right fork %d.\n", id, (id + 1)
% N); }

    printf("Philosopher %d is eating.\n", id);

    sem_post(&forks[id]);
    sem_post(&forks[(id + 1) % N]);

    printf("Philosopher %d put down forks %d and %d.\n",
        id, id, (id + 1) % N);

    return NULL;
}

int main() {
```

```

pthread_t phil[N];
int ids[N];

for (int i = 0; i < N; i++) {
    sem_init(&forks[i], 0, 1);
}

for (int i = 0; i < N; i++) {
    ids[i] = i;
    pthread_create(&phil[i], NULL, philosopher, &ids[i]);
}

for (int i = 0; i < N; i++) {
    pthread_join(phil[i], NULL);
}

return 0;
}

```

Output:

```

Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 3 picked up right fork 4.
Philosopher 0 put down forks 0 and 1.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 is thinking.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 4 picked up left fork 4.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 2 is thinking.

```

Lab Program 5:

Write a C program to simulate:

a) Bankers' algorithm for the purpose of deadlock avoidance b) Deadlock

Detection

a) Bankers' algorithm for the purpose of deadlock avoidance

```
#include <stdio.h>

int main() {
    int n, m, i, j, k, ch;

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    printf("Enter no. of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m], work[m], finish[n], safe[n];

    printf("Enter Allocation Matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&alloc[i][j]);

    printf("Enter Max Matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&max[i][j]);

    printf("Enter Available Resources:\n");
    for(i=0;i<m;i++)
        scanf("%d",&avail[i]);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j]=max[i][j]-alloc[i][j];

    printf("\n1. Safety Algorithm");
    printf("\n2. Resource Request");
```

```
printf("\nEnter choice(1 or 2): ");
```

33

```
scanf("%d",&ch);
```

```
switch(ch) {
```

```
case 1: {
```

```
    for(i=0;i<m;i++)
```

```
        work[i]=avail[i];
```

```
    for(i=0;i<n;i++)
```

```
        finish[i]=0;
```

```
    int count=0;
```

```
    while(count<n) {
```

```
        int found=0;
```

```
        for(i=0;i<n;i++) {
```

```
            if(finish[i]==0) {
```

```
                for(j=0;j<m;j++)
```

```
                    if(need[i][j]>work[j])
```

```
                        break;
```

```
            if(j==m) {
```

```
                for(k=0;k<m;k++)
```

```
                    work[k]+=alloc[i][k];
```

```
                safe[count++]=i;
```

```
                finish[i]=1;
```

```
                found=1;
```

```
            }
```

```
        }
```

```
    }
```

```
    if(found==0)
```

```
        break;
```

```
}
```

```
if(count==n) {
```

```
    printf("\nSAFESTATE\nSafe  
Sequence: "); for(i=0;i<n;i++)
```

```
        printf("P%d",safe[i]);
```

```
}
```

```
else
```

```
    printf("\nUNSAFESTATE");
```

```
break;
```

```
}
```

```

case 2: {
    int p, req[m];
    printf("Enter process number: ");

```

34

```

    scanf("%d",&p);
    printf("Enter Request Vector:\n");
    for(i=0;i<m;i++)
    for(i=0;i<m;i++) {
        if(req[i]>need[p][i]) {
            printf("Error: Exceeded Need");
            return 0;
        }
        if(req[i]>avail[i]) {
            printf("Resources not available"); return 0;
        }
    }
    for(i=0;i<m;i++) {
        avail[i]-=req[i];
        alloc[p][i]+=req[i];
        need[p][i]-=req[i];
    }
    printf("\nRequest Granted\n");
    break;
}
default:
    printf("Invalid Choice");
}
return 0;
}

```

Output:

```
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter Request Matrix:
0 0 0
2 0 2
0 0 0
1 0 1
0 0 2

Enter Available Resources:
0 0 0

P0 can finish.
P2 can finish.
P3 can finish.
P4 can finish.
P1 can finish.

Processes in Deadlock:
No deadlock detected. All processes can finish.
Process returned 0 (0x0)    execution time : 67.833 s
Press any key to continue.
```

b) Deadlock Detection

```
#include<stdio.h>
```

```
int main() {
```

```
    int n,m,i,j,k;
```

```
    printf("Enter number of  
processes: "); scanf("%d",&n);
```

```
    printf("Enter number of resource  
types: "); scanf("%d",&m);
```

```
    int Allocation[n][m],
```

```
    Request[n][m];
```

```
    int Available[m];
```

```
    printf("\nEnter Allocation
```

```
Matrix:\n"); for(i=0;i<n; i++) {
```

```
        for(j=0;j<m; j++) {
```

```
            scanf("%d", &Allocation[i][j]);
```

```
        }
```

```
    }
```

36

```
    printf("\nEnter Request Matri
```

```
x:\n"); for(i = 0; i < n; i++) {
```

```
        for(j = 0; j < m; j++) {
```

```
            scanf("%d",&Request[i][j]);
```

```
        }
```

```
    }
```

```
    printf("\nEnter Available Resource
```

```
s:\n"); for(i = 0; i < m; i++) {
```

```
        scanf("%d",&Available[i]);
```

```
    }
```

```
    int Work[m], Finish[n];
```

```
    for(i = 0; i < m; i++)
```

```
        Work[i] = Available[i];
```

```
    for(i = 0; i < n; i++)
```

```

    Finish[i] = 0;

int count = 0;

while(count < n) {

    int found = 0;

    for(i = 0; i < n; i++) {

        if(Finish[i] == 0) {

            for(j=0;j<m;j++){
                if(Request[i][j]>Work[j])
                    break;
            }

            if(j == m) {

                for(k=0;k<m;k++) {
                    Work[k]+=Allocation[i][k];
                }

                Finish[i]=1;

                found=1;
                count++;
            }
        }
    }

    if(found==0)
        break;
}

int deadlock = 0;

for(i = 0; i < n; i++) {
    if(Finish[i] == 0) {
        deadlock = 1;
        printf("ProcessP%d is
deadlocked.\n", i); }
}

```

```

    }

    if(deadlock == 0)
        printf("\nNoDeadlock Detected.\n");

    return 0;
}

```

Output:

```

Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

System is in SAFE state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2

Enter process number making request: 1
Enter request for resources:
1 0 2

Request can be granted.
New Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2

Process returned 0 (0x0)   execution time : 61.781 s
Press any key to continue.
|

```

Lab Program 6:

Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit

```
#include <stdio.h>

int main() {
    int nb, np, i, j;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);

    int blockSize[nb];

    printf("Enter sizes of %d memory blocks:\n", nb);
    for(i = 0; i < nb; i++) {
        scanf("%d", &blockSize[i]);
    }

    printf("Enter number of processes: ");
    scanf("%d", &np);

    int process[np];

    printf("Enter sizes of %d processes:\n", np);
    for(i = 0; i < np; i++) {
        scanf("%d", &process[i]);
    }

    int firstFit[np], bestFit[np], worstFit[np];

    for(i = 0; i < np; i++) {
        firstFit[i] = -1;
        bestFit[i] = -1;
        worstFit[i] = -1;
    }

    int used1[nb];

    for(i = 0; i < nb; i++)
        used1[i] = 0;
```

```

for(i = 0; i < np; i++) {
    for(j=0;j<nb;j++){
        if(!used1[j]&&blockSize[j] >=
            process[i]) { firstFit[i] = j;
            used1[j] = 1;
            break;
        }
    }
}

int used2[nb];

for(i = 0; i < nb; i++)
    used2[i] = 0;

for(i = 0; i < np; i++) {
    int best = -1;

    for(j=0;j<nb;j++){
        if(!used2[j]&&blockSize[j] >= process[i]) {

            if(best==-1||blockSize[j] <
                blockSize[best]) { best = j;
            }
        }
    }

    if(best != -1) {
        bestFit[i] = best;
        used2[best] = 1;
    }
}

int used3[nb];

for(i = 0; i < nb; i++)
    used3[i] = 0;

for(i = 0; i < np; i++) {
    int worst = -1;

```

```

for(j = 0; j < nb; j++) {
    if(!used3[j]&&blockSize[j] >= process[i]) {

        if(worst==-1||blockSize[j] > blockSize[worst]) {
            worst = j;
        }
    }
}

if(worst != -1) {
    worstFit[i]=worst;
    used3[worst] = 1;
}
}

printf("\n---FirstFit---\n");
printf("ProcessNo.\tProcess Size\tBlock No.\n");

for(i = 0; i < np; i++) {
    if(firstFit[i] != -1)
        printf("%d\t%d\t%d\n", i + 1, process[i],
            firstFit[i] + 1); else
        printf("%d\t%d\t\tNot Allocated\n", i + 1,
process[i]); }

printf("\n---BestFit---\n");
printf("ProcessNo.\tProcess Size\tBlock No.\n");

for(i = 0; i < np; i++) {
    if(bestFit[i] != -1)
        printf("%d\t%d\t%d\n", i + 1, process[i],
            bestFit[i] + 1); else
        printf("%d\t%d\t\tNot Allocated\n", i + 1,
process[i]); }

printf("\n---WorstFit---\n");
printf("ProcessNo.\tProcess Size\tBlock No.\n");

for(i = 0; i < np; i++) {
    if(worstFit[i] != -1)

```

```

        printf("%d\t%d\t%d\n", i + 1, process[i],
worstFit[i] + 1); else

```

41

```

        printf("%d\t%d\tNot Allocated\n", i + 1,
process[i]); }

```

```

return 0;
}

```

Output:

```

Enter number of memory blocks: 6
Enter sizes of 6 memory blocks:
300 600 350 200 750 125

Enter number of processes: 5
Enter sizes of 5 processes:
115 500 358 200 375

--- First Fit ---
Process No.      Process Size      Block No.
1                115                1
2                500                2
3                358                5
4                200                3
5                375                Not Allocated

--- Best Fit ---
Process No.      Process Size      Block No.
1                115                6
2                500                2
3                358                5
4                200                4
5                375                Not Allocated

--- Worst Fit ---
Process No.      Process Size      Block No.
1                115                5
2                500                2
3                358                Not Allocated
4                200                3
5                375                Not Allocated

Process returned 0 (0x0)   execution time : 58.930 s
Press any key to continue.

```

Lab Program 7:

Write a C program to simulate page replacement algorithms a) FIFO b) LRU c) Optimal

```
#include <stdio.h>
```

```
int fifo(int p[], int n, int f) {  
    int fr[10], i, j, k = 0, fault = 0, found;
```

```
    for(i = 0; i < f; i++) fr[i] = -1;
```

```
    for(i = 0; i < n; i++) {  
        found = 0;  
        for(j = 0; j < f; j++)  
            if(fr[j] == p[i]) found = 1;
```

```
        if(!found) {  
            fr[k] = p[i];  
            k = (k + 1) % f;  
            fault++;
```

```
        }
```

```
    }
```

```
    return fault;
```

```
}
```

```
int lru(int p[], int n, int f) {  
    int fr[10], time[10], i, j, pos, min, fault = 0;
```

```
    for(i = 0; i < f; i++) {  
        fr[i] = -1;  
        time[i] = -1;
```

```
    }
```

```
    for(i = 0; i < n; i++) {  
        int found = 0;
```

```
        for(j = 0; j < f; j++) {  
            if(fr[j] == p[i]) {  
                found = 1;
```

```
                time[j] = i;
```

```
            }
```

```
        }
```

```

if(!found) {
    min=time[0];
    pos = 0;

    for(j=1;j<f;j++) {
        if(time[j]<min) {
            min=time[j];
            pos = j;
        }
    }
    fr[pos] = p[i];
    time[pos] = i;
    fault++;
}
}
return fault;
}

```

```

intoptimal(intp[],intn,int f) {
    intfr[10],i,j,k,pos,farthest,
    fault = 0;

    for(i = 0; i < f; i++)
        fr[i] = -1;

    for(i = 0; i < n; i++) {
        int found = 0;

        for(j=0;j<f;j++){
            if(fr[j] == p[i]) {
                found=1;
                break;
            }
        }

        if(!found) {
            pos = -1;
            farthest = -1;

            for(j=0;j<f;j++) {

```

```
intnextUse=-1;
```

44

```
for(k = i + 1; k < n; k++) {  
    if(fr[j] == p[k]) {  
        nextUse = k;  
        break;  
    }  
}
```

```
if(nextUse == -1) {  
    pos = j;  
    break;  
}
```

```
if(nextUse > farthest) {  
    farthest = nextUse;  
    pos = j;  
}  
}
```

```
fr[pos] = p[i];  
fault++;  
}  
}
```

```
return fault;  
}
```

```
int main() {  
    int p[20], n, f, i;  
  
    printf("Enter number of pages: ");  
    scanf("%d", &n);  
  
    printf("Enter pages: ");  
    for(i = 0; i < n; i++)  
        scanf("%d", &p[i]);  
  
    printf("Enter number of frames: ");  
    scanf("%d", &f);
```

```

printf("\nFIFO Faults = %d", fifo(p, n, f));
printf("\nLRU Faults = %d", lru(p, n, f));
printf("\nOptimal Faults = %d", optimal(p, n,
f)); }

```

45

Output:

```

Enter number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3

FIFO Page Replacement:
Page 1 -> 1 -1 -1
Page 2 -> 1 2 -1
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 4 1 3
Page 2 -> 4 1 2
Page 5 -> 5 1 2
Page 3 -> 5 3 2
Page 4 -> 5 3 4
Total Page Faults = 9

LRU Page Replacement:
Page 1 -> 1 -1 -1
Page 2 -> 1 2 -1
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 4 1 3
Page 2 -> 4 1 2
Page 5 -> 5 1 2
Page 3 -> 3 1 2
Page 4 -> 3 4 2
Page 5 -> 3 4 5
Total Page Faults = 10

Optimal Page Replacement:
Page 1 -> 1 -1 -1
Page 2 -> 1 2 -1
Page 3 -> 1 2 3
Page 4 -> 1 2 4
Page 5 -> 1 2 5
Page 3 -> 3 2 5
Page 4 -> 4 2 5
Total Page Faults = 7

Process returned 0 (0x0)   execution
Press any key to continue.
|

```

